Rapport Technique sur l'Écosystème OpenClaw : Intégration Avancée avec Visual Studio Code, Claude Code et les Agents IA Multi-Modèles

L'évolution du paysage de l'intelligence artificielle en 2026 a marqué un tournant décisif, passant de simples interfaces de discussion réactives à des systèmes agentiques autonomes capables de s'intégrer profondément dans les environnements de développement locaux. Au cœur de cette transformation se trouve l'écosystème OpenClaw, un projet qui, bien que partageant un nom unique, se divise en deux branches technologiques distinctes mais complémentaires : un framework d'assistants IA sophistiqué et une réimplémentation moderne de moteur de jeu pour le classique de 1997, Captain Claw. La maîtrise de cet écosystème exige une compréhension rigoureuse de la configuration de Visual Studio Code comme centre de commande, ainsi que l'orchestration de modèles de langage de pointe issus d'Anthropic, d'OpenAI et de DeepSeek. Ce rapport examine les protocoles techniques nécessaires pour déployer, configurer et optimiser ces outils afin de créer un environnement de développement local de haute fidélité, capable de gérer des tâches complexes allant de la génération de code autonome à la rétro-ingénierie logicielle.

Architecture et Fondations Techniques des Projets OpenClaw

Une analyse approfondie des dépôts officiels révèle une convergence nominale qui peut induire en erreur les développeurs lors de la phase initiale de configuration. Il est impératif de distinguer le framework d'agents IA, géré par l'organisation openclaw sur GitHub, de la réimplémentation du moteur de jeu maintenue principalement dans le dépôt pjasicek/OpenClaw.[1] Cette distinction n'est pas simplement sémantique ; elle définit les exigences système, les piles technologiques et les flux de travail opérationnels.

La Nature Duale des Référentiels OpenClaw

Le framework d'assistant personnel OpenClaw, initialement lancé sous le nom de « Clawdbot » fin 2025 par le développeur autrichien Peter Steinberger, a connu une expansion rapide pour devenir une architecture de contrôle d'agents distribués.[4] Construit sur Node.js, ce système fonctionne comme un service de longue durée qui relie des modèles de langage tels que Claude d'Anthropic ou GPT-5.4 d'OpenAI à la machine locale de l'utilisateur et à des applications de messagerie.[4] Son architecture repose sur quatre piliers fondamentaux : la passerelle (Gateway), qui sert de plan de contrôle toujours actif ; les canaux (Channels), permettant des intégrations natives avec plus de 50 plateformes de messagerie ; les

compétences (Skills), des modules add-ons gérés via ClawHub ; et les outils (Tools), qui fournissent un accès au niveau système pour l'automatisation du navigateur, l'exécution du shell et la gestion des fichiers.[4]

À l'opposé, le moteur de jeu OpenClaw est une réimplémentation multiplateforme en C++17 du moteur WAP32 original utilisé par Monolith Productions.[5] Ce projet se concentre sur la préservation logicielle, permettant au jeu de 1997 de fonctionner nativement sur les systèmes d'exploitation modernes avec des fonctionnalités améliorées telles que le support des écrans larges et des contrôleurs de jeu actuels.[5] Cette branche nécessite l'acquisition légale des actifs originaux, notamment le fichier CLAW.REZ, car le moteur lui-même ne contient aucune donnée protégée par le droit d'auteur.[2]

Caractéristique	**Assistant IA OpenClaw**	**Moteur de Jeu OpenClaw**
Organisation GitHub	openclaw	pjasicek
Langage Principal	TypeScript / Node.js	C++17
Objectif Principal	Automatisation de tâches	Préservation de jeu rétro
Dépendance Critique	Clés API LLM	Archive CLAW.REZ
Interface Utilisateur	Tableau de bord Web (18789)	Fenêtre native SDL2
Licence	MIT	GPL-3.0

1

Déploiement et Configuration de l'Environnement Local

Le succès d'une intégration OpenClaw repose sur une préparation méticuleuse de

l'environnement hôte. Pour le framework d'assistants IA, les spécifications minimales recommandent un processeur de 2 à 4 cœurs, 8 Go de RAM et un accès Internet constant pour la gestion des dépendances.[8] L'utilisation de Node.js version 22 ou supérieure est indispensable, la version 24 étant fortement conseillée pour garantir la stabilité du daemon de la passerelle.[8]

Installation du Framework d'Agents OpenClaw

Le processus d'installation standard commence par le clonage du dépôt officiel. Les développeurs privilégient souvent le gestionnaire de paquets pnpm en raison de son efficacité dans la gestion des liens symboliques et de l'espace disque. Après avoir cloné le référentiel avec git clone https://github.com/openclaw/openclaw.git, l'exécution de pnpm install télécharge l'ensemble des dépendances nécessaires aux services backend et à l'interface utilisateur du tableau de bord.[8] La compilation du code TypeScript est ensuite effectuée via les commandes pnpm build et pnpm ui:build.[8]

Une fois l'application compilée, l'établissement d'un lien global via pnpm link --global permet d'accéder à la commande openclaw depuis n'importe quel point du système.[8] L'étape cruciale de l'onboarding, déclenchée par openclaw onboard --install-daemon, guide l'utilisateur dans la configuration des clés API, des services d'arrière-plan et des paramètres de l'espace de travail.[8] Ce processus installe également les services système nécessaires pour assurer la persistance de la passerelle, que ce soit via systemd sur Linux ou des tâches planifiées sur Windows.[9]

Compilation et Build du Moteur de Jeu C++

Pour les développeurs souhaitant exploiter le moteur de jeu pjasicek/OpenClaw à des fins de recherche ou de modding, la compilation à partir des sources offre le plus grand contrôle. Sur les systèmes basés sur Debian ou Ubuntu, l'installation des outils de construction essentiels et des bibliothèques de développement SDL2 est un prérequis strict. La commande sudo apt-get install build-essential cmake libsdl2-dev libsdl2-image-dev libsdl2-mixer-dev libsdl2-ttf-dev libtinyxml2-dev git prépare l'environnement pour le processus de build Unix standard.[6]

Le flux de travail CMake implique la création d'un répertoire de construction séparé pour maintenir la propreté de l'arbre source. En exécutant cmake.. suivi de make -j$(nproc), le système utilise l'ensemble des cœurs de processeur disponibles pour compiler et lier le binaire openclaw.[10] Il est impératif que le fichier CLAW.REZ soit placé dans le même répertoire que l'exécutable généré, car le moteur analyse son répertoire de travail pour localiser les archives d'actifs au démarrage.[5]

Intégration Profonde avec Visual Studio Code

Visual Studio Code (VS Code) s'est imposé comme le centre névralgique pour l'orchestration des agents OpenClaw, offrant une interface graphique et un terminal intégré qui facilitent la gestion des flux de travail agentiques. L'utilisation conjointe de l'extension Claude Code et du plugin OpenAI Codex permet de créer un environnement où plusieurs modèles collaborent sur

le même projet.

Configuration de Claude Code comme Orchestrateur Local

Claude Code fonctionne comme un assistant de programmation IA basé sur un terminal et une extension VS Code, conçu pour comprendre l'intégralité d'une base de code sans sélection manuelle de contexte.[13] Son installation dans VS Code nécessite la version 1.98.0 ou supérieure de l'éditeur.[15] Une fois installée, l'extension se manifeste par une icône « Spark » dans la barre d'activité ou la barre d'outils de l'éditeur.[15]

L'un des avantages majeurs de Claude Code réside dans sa gestion des modes de permission. Le mode « Plan » est particulièrement sophistiqué : Claude décrit sa stratégie dans un document markdown complet, permettant au développeur d'ajouter des commentaires en ligne pour orienter l'IA avant qu'elle ne commence à modifier les fichiers.[15] Pour les environnements de bac à sable isolés, le paramètre github.copilot.chat.claudeAgent.allowDangerouslySkipPermissions permet de contourner les vérifications de permission, accélérant ainsi les cycles de développement autonomes.[16]

Exploitation d'OpenAI Codex pour la Délégation de Tâches

Parallèlement à Claude Code, l'agent OpenAI Codex (anciennement connu sous le nom de Codex) propose un environnement cloud pour déléguer des tâches de développement de bout en bout.[13] Contrairement à l'approche locale de Claude Code, Codex peut exécuter des tâches de manière asynchrone dans des bacs à sable cloud isolés, préchargés avec le dépôt de l'utilisateur.[13]

L'intégration de Codex dans VS Code se fait via l'extension officielle, qui permet de se connecter avec un compte ChatGPT Pro ou une clé API OpenAI.[17] Dans le panneau Codex, l'utilisateur peut initier des sessions locales ou cloud, permettant à l'agent de lire des fichiers, d'exécuter des commandes et d'appliquer des modifications de manière autonome.[16] L'aspect le plus puissant de cette configuration est le plugin Codex pour Claude Code (codex@openai-codex), qui permet à Claude de déléguer des tâches spécifiques, comme une revue de code contradictoire ou un sprint d'implémentation, à Codex sans quitter le flux de travail initial.[19]

Agent	**Philosophie de Travail**	**Environnement d'Exécution**	**Force Majeure**
Claude Code	Développeur dans la boucle	Local / Terminal intégré	Analyse profonde du codebase

OpenAI Codex	Délégation autonome	Cloud Sandbox / Local	Tâches asynchrones complexes
OpenClaw AI	Plan de contrôle agentique	Passerelle self-hosted	Multi-canal et gestion de sessions

13

Configuration des Clés API et Multi-Modèles : OpenAI, Claude et DeepSeek

La puissance de l'écosystème OpenClaw réside dans sa capacité à être alimenté par divers fournisseurs de modèles via des clés API personnalisées. Cette approche, souvent appelée « Bring Your Own Key » (BYOK), permet aux développeurs de choisir le modèle le plus adapté à la tâche en cours tout en optimisant les coûts.

Routage vers DeepSeek via les API compatibles Anthropic

DeepSeek, avec ses modèles V3 et R1, est devenu une alternative de choix pour les tâches de codage en raison de ses performances élevées et de son coût réduit.[23] Pour configurer Claude Code afin qu'il communique avec DeepSeek, il est nécessaire de détourner les requêtes API standards d'Anthropic vers la passerelle DeepSeek.[24]

Ce processus s'effectue principalement par la définition de variables d'environnement dans le shell ou dans les paramètres de VS Code. La variable ANTHROPIC_BASE_URL doit pointer vers https://api.deepseek.com/anthropic, tandis que ANTHROPIC_AUTH_TOKEN doit contenir la clé API DeepSeek.[6] Il est également recommandé de définir ANTHROPIC_MODEL sur deepseek-chat pour s'assurer que le client démarre avec le bon modèle.[24] Pour les sessions VS Code, ces variables peuvent être injectées via le paramètre claudeCode.environmentVariables dans le fichier settings.json.[14]

Utilisation de LiteLLM comme Couche de Proxy Unifiée

Pour une gestion plus fluide de plusieurs fournisseurs, l'utilisation de LiteLLM comme proxy est une stratégie avancée. LiteLLM agit comme une couche de traduction qui reçoit les requêtes au format Anthropic Messages API et les convertit pour le fournisseur cible (OpenAI, Gemini, ou DeepSeek).[28] Après avoir installé LiteLLM via pip install 'litellm[proxy]', le développeur configure un fichier config.yaml définissant les modèles virtuels et les clés API correspondantes.[28]

Une fois le proxy lancé sur http://0.0.0.0:4000, il suffit de définir ANTHROPIC_BASE_URL sur cette adresse locale pour que Claude Code puisse accéder à une flotte de modèles non-Anthropic.[28] Cette configuration permet de basculer instantanément d'un modèle GPT-4o à un modèle DeepSeek ou Gemini simplement en changeant le nom du modèle dans la commande de lancement : claude --model gpt-4o.[28]

Protocoles d'Authentification et Priorités

Il est crucial de comprendre la hiérarchie d'authentification pour éviter les facturations inattendues. Claude Code donne la priorité aux clés API définies comme variables d'environnement (ANTHROPIC_API_KEY) par rapport aux abonnements authentifiés via le web.[29] Si une clé est présente dans l'environnement, le système utilisera le compte associé à cette clé en mode paiement à l'usage, même si l'utilisateur est connecté à un abonnement Claude Pro.[30] La vérification de la configuration active se fait via la commande /status à l'intérieur de Claude Code.[25]

Fournisseur	**Variable de Base URL**	**Jeton d'Authentification**	**Modèle Recommandé**
Anthropic	Native (automatique)	Connexion Web / ANTHROPIC_API_KEY	Claude 3.5 Sonnet
DeepSeek	https://api.deepseek.com/anthropic	ANTHROPIC_AUTH_TOKEN	deepseek-chat
OpenAI (Proxy)	http://localhost:4000 (LiteLLM)	LITELLM_MASTER_KEY	gpt-4o
Google Gemini	https://generativelanguage.googleapis.com/v1beta/openai/	ANTHROPIC_AUTH_TOKEN	gemini-2.5-flash

Flux de Travail et Outils de Développement Multi-Agent

L'intégration réussie de ces agents dans Visual Studio Code transforme radicalement le cycle de développement. L'utilisation du protocole Model Context Protocol (MCP) et des outils de délégation permet de créer des flux de travail automatisés pour des projets complexes.

Extension des Capacités via le protocole MCP

Le Model Context Protocol est devenu le standard pour connecter les agents IA à des sources de données et des outils externes.[32] En intégrant des serveurs MCP comme Composio, les agents OpenClaw peuvent accéder à plus de 20 000 outils couvrant 870 applications.[32] L'ajout d'un serveur MCP à Claude Code se fait via la commande claude mcp add, ce qui permet à l'IA d'interagir avec des services comme GitHub, Slack ou des bases de données SQL directement depuis le terminal VS Code.[33]

Cette extensibilité est illustrée par le fonctionnement des « Skills » dans OpenClaw. Chaque compétence est définie par un fichier SKILL.md qui expose des outils spécifiques à l'agent.[4] Par exemple, une compétence de recherche web peut être installée via openclaw plugins install clawhub:websearch, permettant à l'agent de consulter la documentation en ligne pour résoudre des erreurs de compilation C++ en temps réel.[4]

Collaboration entre Claude Code et OpenAI Codex

Dans un projet de grande envergure, comme la maintenance du moteur de jeu OpenClaw, les agents peuvent être utilisés de manière complémentaire. Claude Code excelle dans l'analyse de la base de code locale et l'exécution de commandes système, tandis que Codex est idéal pour les tâches de révision de code à grande échelle.[13]

Un flux de travail efficace consiste à demander à Claude Code de planifier et d'implémenter une nouvelle fonctionnalité, comme l'ajout d'un nouveau système de particules au moteur C++.[13] Une fois les changements proposés, le développeur peut invoquer le plugin Codex (/codex:review) pour obtenir un second avis d'un modèle différent.[19] Cette boucle de rétroaction multi-modèle réduit considérablement les erreurs logiques et les régressions, car chaque modèle apporte ses propres forces de raisonnement.[20]

Résolution des Problèmes Techniques et Maintenance

Le déploiement de solutions aussi complexes que les agents OpenClaw et les moteurs de jeu C++ s'accompagne inévitablement de défis techniques. Une compréhension des erreurs courantes et de leurs solutions est essentielle pour maintenir un environnement de développement productif.

Débogage des erreurs de compilation C++ avec l'IA

La compilation du moteur pjasicek/OpenClaw peut échouer en raison de dépendances manquantes ou de configurations de chemin incorrectes. L'utilisation de Claude Code dans le terminal intégré de VS Code permet de diagnostiquer ces erreurs instantanément. En fournissant l'erreur du compilateur à l'agent (ex: « @src/Main.cpp fails to compile with 'SDL.h' not found »), Claude peut suggérer l'installation des paquets -dev manquants ou modifier le fichier CMakeLists.txt pour inclure les répertoires appropriés.[15]

Sur Windows, les erreurs liées aux fichiers DLL manquants (SDL2.dll ou MSVCP140.dll) sont fréquentes. Claude peut automatiser le téléchargement et le placement de ces bibliothèques dans le dossier de l'exécutable, ou guider l'utilisateur dans l'installation du package redistribuable Microsoft Visual C++.[10]

Gestion de la mémoire et stabilité des extensions

Les assistants IA agentiques peuvent consommer une quantité importante de mémoire, ce qui peut parfois entraîner le crash de l'hôte d'extension de VS Code.[37] Pour pallier ce problème, il est recommandé d'augmenter la limite de mémoire allouée à VS Code via le flag --max-memory=8192 lors du lancement.[37] De plus, le redémarrage périodique de l'hôte d'extension (« Developer: Restart Extension Host ») permet de libérer la mémoire accumulée sans fermer l'éditeur.[37]

Pour garantir une stabilité maximale, l'exécution de Claude Code directement dans le terminal natif (plutôt que dans le panneau latéral de VS Code) est souvent préférable pour les tâches de longue durée, car cela isole le processus de l'IA de l'interface graphique de l'éditeur.[37]

Symptôme	Cause Probable	Solution Technique
Crash immédiat du moteur OpenClaw	Fichier CLAW.REZ manquant	Vérifier la présence et le nom du fichier dans le répertoire racine.[5]
"Unable to connect to API"	Conflit ANTHROPIC_BASE_URL	Vérifier les variables d'environnement avec echo $ANTHROPIC_BASE_URL.[25]
Lenteur d'inférence (90%+)	Invalidation du KV Cache	Désactiver le header d'attribution via CLAUDE_CODE_ATTRIBUTION_HEADER=0.[38]
Erreur 429 (Rate Limit)	Dépassement des quotas API	Basculer vers un autre fournisseur (DeepSeek ou

		OpenAI) via LiteLLM.[28]

5

Sécurité, Confidentialité et Isolation

L'utilisation d'agents IA disposant d'un accès au système de fichiers et au shell pose des défis de sécurité majeurs. L'isolation des agents est une priorité absolue pour protéger les données sensibles de l'utilisateur.

Stratégies de Sandboxing et Docker

Il est fortement recommandé d'exécuter la passerelle OpenClaw et ses agents dans des environnements conteneurisés.[4] L'utilisation de Docker permet de restreindre l'accès de l'IA à des répertoires spécifiques de l'hôte, empêchant ainsi la lecture accidentelle de fichiers de configuration sensibles ou de clés SSH.[4] Pour les développeurs travaillant sur des projets d'entreprise, le déploiement sur un VPS dédié offre une couche supplémentaire d'isolation par rapport à la machine de travail principale.[8]

Gestion des données et respect de la vie privée

Bien que Claude Code et Codex opèrent localement, ils transmettent des extraits de code aux serveurs d'Anthropic ou d'OpenAI pour le traitement.[13] Pour les utilisateurs soucieux de la confidentialité, l'utilisation de modèles locaux via LM Studio ou Ollama est possible en redirigeant ANTHROPIC_BASE_URL vers un serveur llama.cpp local.[26] Cette approche garantit que les données ne quittent jamais la machine de l'utilisateur, au prix d'une exigence matérielle plus élevée (notamment en termes de VRAM GPU).[31]

Conclusion sur l'Écosystème Agentique OpenClaw

L'intégration d'OpenClaw avec Visual Studio Code, Claude Code et OpenAI Codex représente le summum du développement assisté par l'IA en 2026. Cette synergie permet non seulement de moderniser des logiciels anciens, comme le démontre le moteur de jeu OpenClaw, mais aussi de transformer radicalement la productivité des développeurs grâce à des agents capables d'agir de manière autonome et coordonnée.

La clé d'une utilisation réussie réside dans la flexibilité du routage des modèles. En exploitant des passerelles comme DeepSeek et des proxies comme LiteLLM, les développeurs peuvent s'affranchir des limitations d'un fournisseur unique et construire des flux de travail optimisés pour la performance, le coût et la sécurité. Alors que les agents IA continuent d'évoluer vers une compréhension toujours plus fine des systèmes complexes, la maîtrise de ces configurations de contrôle et d'orchestration deviendra une compétence fondamentale pour tout ingénieur logiciel moderne. L'écosystème OpenClaw, par sa nature open-source et sa structure modulaire, offre la plateforme idéale pour explorer cette nouvelle ère de la

programmation agentique.

Sources des citations

1. OpenClaw - GitHub, consulté le avril 10, 2026, https://github.com/openclaw
2. The OpenClaw Game Engine: A Deep Dive into the Modern Rebirth of a 90s Classic, consulté le avril 10, 2026, https://skywork.ai/skypage/en/openclaw-game-engine-modern-rebirth/2037081330265362432
3. Unveiling the Classic: openclaw game what is it? - Skywork, consulté le avril 10, 2026, https://skywork.ai/skypage/en/openclaw-game-overview/2038535914843357184
4. The Ultimate Guide to the OpenClaw GitHub Official Repository 2026 - Skywork, consulté le avril 10, 2026, https://skywork.ai/skypage/en/ultimate-guide-openclaw-github/2038544722909990912
5. The Ultimate OpenClaw Game Installation Guide: Engine Setup and Analysis - Skywork.ai, consulté le avril 10, 2026, https://skywork.ai/skypage/en/openclaw-game-installation-guide/2038531779259613184
6. The Ultimate OpenClaw Game Installation Guide: Bridging Retro Gaming and AI - Skywork, consulté le avril 10, 2026, https://skywork.ai/skypage/en/ultimate-openclaw-game-installation/2038583622081789952
7. Openclaw: Unleashing the Next Generation of Open-Source... - Petronella Technology Group, consulté le avril 10, 2026, https://petronellatech.com/blog/openclaw-unleashing-the-next-generation-of-open-source-power/
8. OpenClaw GitHub Guide: Installation, Setup and Troubleshooting - Bluehost, consulté le avril 10, 2026, https://www.bluehost.com/blog/openclaw-github-guide/
9. Install - OpenClaw Docs, consulté le avril 10, 2026, https://docs.openclaw.ai/install
10. How to Install OpenClaw Locally: A Comprehensive Technical Guide - H3sync, consulté le avril 10, 2026, https://h3sync.com/blog/how-to-install-openclaw-locally-a-comprehensive-technical-guide/
11. How to Install OpenClaw Locally: The Definitive Technical Resource & Setup Guide, consulté le avril 10, 2026, https://h3sync.com/blog/how-to-install-openclaw-locally-the-definitive-technical-resource-setup-guide/
12. How to Install OpenClaw Locally: The Ultimate Technical Setup Guide - H3sync, consulté le avril 10, 2026, https://h3sync.com/blog/how-to-install-openclaw-locally-the-ultimate-technical-setup-guide/
13. Claude Code vs OpenAI Codex: which is better in 2026? | Blog - Northflank,

consulté le avril 10, 2026, https://northflank.com/blog/claude-code-vs-openai-codex
14. Claude Code--ModelArk-Byteplus, consulté le avril 10, 2026, https://docs.byteplus.com/en/docs/ModelArk/2277823
15. Use Claude Code in VS Code - Claude Code Docs, consulté le avril 10, 2026, https://code.claude.com/docs/en/vs-code
16. Third-party agents in Visual Studio Code, consulté le avril 10, 2026, https://code.visualstudio.com/docs/copilot/agents/third-party-agents
17. Quickstart – Codex - OpenAI Developers, consulté le avril 10, 2026, https://developers.openai.com/codex/quickstart
18. Integrating OpenAI Codex into VS Code using ChatGPT | by Alesson Viana - Medium, consulté le avril 10, 2026, https://medium.com/@alesson.viana/integrating-openai-codex-into-vs-code-using-chatgpt-a17c2437e8c7
19. Introducing Codex Plugin for Claude Code - OpenAI Developer Community, consulté le avril 10, 2026, https://community.openai.com/t/introducing-codex-plugin-for-claude-code/1378186
20. When Rivals Collaborate: Installing OpenAI's Codex Plugin in Claude Code | by Mark Chen | Mar, 2026 | Medium, consulté le avril 10, 2026, https://medium.com/@markchen69/when-rivals-collaborate-installing-openais-codex-plugin-in-claude-code-5d3e503ce493
21. Plan with Claude and hand off to Codex to implement in a worktree : r/ClaudeCode - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/ClaudeCode/comments/1sbymxh/plan_with_claude_and_hand_off_to_codex_to/
22. How to Use Codex: A Comprehensive Guide to OpenAI's Revolutionary AI Coding Agent, consulté le avril 10, 2026, https://www.ai.cc/blogs/how-to-use-openai-codex-ai-coding-guide/
23. I built a VS Code extension to route Claude Code/Copilot through DeepSeek V3 & R1 via OpenGPU (RelayCode) - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/DeepSeek/comments/1s4ny9f/i_built_a_vs_code_extension_to_route_claude/
24. DeepSeek Anthropic API in Claude Code - DeepSeek AI, consulté le avril 10, 2026, https://chat-deep.ai/guide/anthropic_api/
25. How I Set Up Claude Code for Free (No Subscription, No Credit Card) — and What I Learned Along the Way | by Prince Arora, consulté le avril 10, 2026, https://prince-arora-aws.medium.com/how-i-set-up-claude-code-for-free-no-subscription-no-credit-card-and-what-i-learned-along-the-2cba880682a2
26. Use claude-code for free in the terminal, VSCode extension or via discord like openclaw - GitHub, consulté le avril 10, 2026, https://github.com/Rishurajgautam24/free-claude-code
27. Claude Code + Microsoft Foundry: Enterprise AI Coding Agent Setup | All things Azure, consulté le avril 10, 2026, https://devblogs.microsoft.com/all-things-azure/claude-code-microsoft-foundry-

enterprise-ai-coding-agent-setup/
28. Use Claude Code with Non-Anthropic Models | liteLLM, consulté le avril 10, 2026, https://docs.litellm.ai/docs/tutorials/claude_non_anthropic_models
29. Claude Code credentials or API key when using Claude Agent SDK? - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/ClaudeCode/comments/1ogzcxa/claude_code_credentials_or_api_key_when_using/
30. Managing API key environment variables in Claude Code | Claude Help Center, consulté le avril 10, 2026, https://support.claude.com/en/articles/12304248-managing-api-key-environment-variables-in-claude-code
31. How to Use a Different LLM with Claude Code (2025 Guide) - Morph, consulté le avril 10, 2026, https://morphllm.com/use-different-llm-claude-code
32. How to integrate DeepSeek MCP with Codex - Composio, consulté le avril 10, 2026, https://composio.dev/toolkits/deepseek/framework/codex
33. DeepSeek MCP Integration with Claude Code - Composio, consulté le avril 10, 2026, https://composio.dev/toolkits/deepseek/framework/claude-code
34. Agent SDK overview - Claude Code Docs, consulté le avril 10, 2026, https://code.claude.com/docs/en/agent-sdk/overview
35. Anyone else using Claude Code + Codex together? way to automise my workflow?, consulté le avril 10, 2026, https://www.reddit.com/r/ClaudeCode/comments/1rh0kuo/anyone_else_using_claude_code_codex_together_way/
36. my vs code not working for c++ compile and show errors like .exe missing - Microsoft Learn, consulté le avril 10, 2026, https://learn.microsoft.com/en-in/answers/questions/5547242/my-vs-code-not-working-for-c-compile-and-show-erro
37. Claude Code + VSCode: Why It Crashes and How to Fix It | by Gaurav Chaulagain | Medium, consulté le avril 10, 2026, https://medium.com/@gauravchaulagain/claude-code-vscode-why-it-crashes-and-how-to-fix-it-9179664dc63d
38. How to Run Local LLMs with Claude Code | Unsloth Documentation, consulté le avril 10, 2026, https://unsloth.ai/docs/basics/claude-code
39. How to get deepseek in a codex or Claude code style set up - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/DeepSeek/comments/1qe3e8p/how_to_get_deepseek_in_a_codex_or_claude_code/
40. OpenClaw – Moltbot Renamed Again - Hacker News, consulté le avril 10, 2026, https://news.ycombinator.com/item?id=46820783
41. Top 5 Best OpenClaw Hosting Providers (2026 Edition) - Web Hosting | MilesWeb, consulté le avril 10, 2026, https://www.milesweb.com/blog/hosting/vps/best-openclaw-hosting-providers/
42. How to Use DeepSeek R1 for Free in Visual Studio Code with Cline or Roo Code, consulté le avril 10, 2026, https://dev.to/dwtoledo/how-to-use-deepseek-r1-for-free-in-visual-studio-code

-with-cline-or-roo-code-3an9